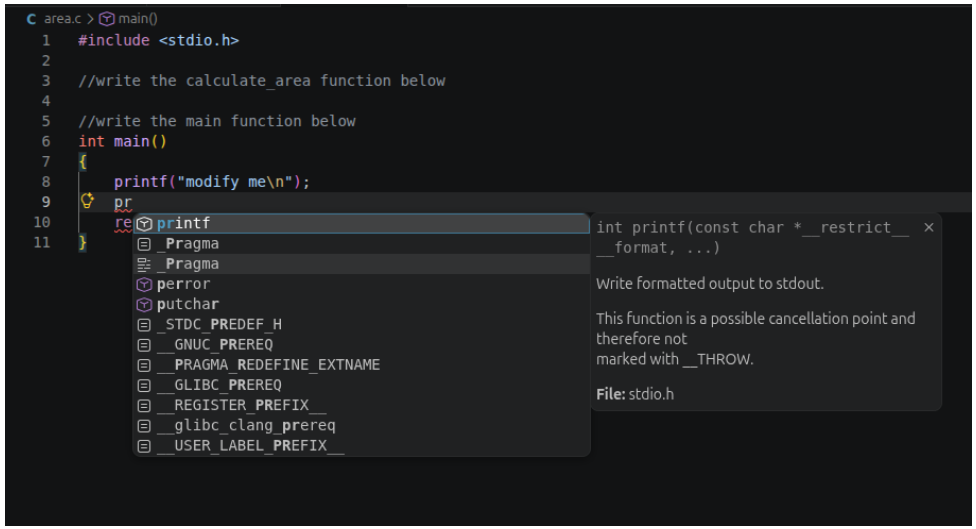
Lab 5: Structures and Dynamic Allocation

Matteo Fasiolo

VSCode set up (ignore if you have already done this)

VScode comes with auto-suggestion features:



These are useful for experienced programmers, but are detrimental for learners.

The exam is pencil-and-paper, so it's best to disable all these features.

The next slide explains how.

Open VSCode and press Ctrl+Shift+P (Cmd+Shift+P on Macs).

In the text box, type `Preferences: Open User Settings (JSON)`.

Copy-paste this code (including brackets) into that file:

```
{
"chat.disableAIFeatures": true,
"editor.inlineSuggest.enabled": false,
"editor.quickSuggestions": false,
"editor.suggestOnTriggerCharacters": false,
"editor.wordBasedSuggestions": "off",
"editor.parameterHints.enabled": false,
"editor.snippetSuggestions": "none",
"editor.tabCompletion": "off",
"editor.acceptSuggestionOnEnter": "off",
"editor.acceptSuggestionOnCommitCharacter": false,
"C_Cpp.autocomplete": "disabled"
}
```

Lab outline

Complete the four homeworks in order:

HMW1. use the members of a simple structure;

HMW2. allocate vector elements using `malloc`;

HMW3. store a fixed-size array inside a structure;

HMW4. store a pointer to a dynamically allocated array inside a structure.

Homeworks 2–4 perform the same vector calculation using different representations.

Homework 1: a student structure

Open `student.c`.

1. Inspect the definition of `struct student` and the variable `matteo`.
2. Complete **TODO 1** so that `print_student` prints every member.
3. For **TODO 2**, create a second student using an initializer list:
   - id $= 2025$, year $= 1$ and mark $= 64.5$
4. Call `print_student` for both students.

Expected output:

```
Student 1024: year 1, mark 70.0
Student 2048: year 2, mark 64.5
```

Homework 2: vector operations with `malloc`

Open `vector_ops.c`. Here each array and its length are passed separately.

1. Complete **TODO 1** in `print_array`.
2. Complete **TODO 2** in `dot`. You may assume that the two arrays have the same length.
3. For **TODO 3**, use `malloc` to allocate `length` doubles for each of `u` and `v`.

Do not run the program until both pointers have been assigned their allocated arrays!

4. For **TODO 4**, initialise

```
u = [1, 2, 3]
v = [4, 5, 6]
```

5. For **TODO 5**, print both arrays and their dot product.
6. For **TODO 6**, release both allocations before returning from `main`.

Expected output:

```
u: 1.00 2.00 3.00
v: 4.00 5.00 6.00
dot product: 32.00
```

Homework 3: vectors using structures fixed-size arrays

Open `vector_fixed.c`.

The array is now stored inside the structure:

```
struct vector {
    int length;
    double elements[3];
};
```

1. Complete **TODO 1** in `print_vector`.
2. Complete **TODO 2** in `dot`.
3. For **TODO 3**, initialise `u` and `v` with the same values as Homework 2.
4. For **TODO 4**, print both vectors and their dot product.

Homework 4: a dynamic vector structure

Open `vector_ops2.c`.

The structure now stores a pointer to a separately allocated array:

```
struct vector2 {
    int length;
    double *elements;
};
```

1. Complete **TODO 1** in `print_vector2`.
2. Complete **TODO 2** in `dot`.
3. Run-time input determines the length of both vectors.

Homework 4: allocate, initialise and use

4. For **TODO 3**, allocate the element arrays for `u` and `v`.
5. For **TODO 4**, initialise their elements using

```
u.elements[i] = i + 1.0;
v.elements[i] = i + 4.0;
```

6. For **TODO 5**, print both vectors and their dot product.
7. For **TODO 6**, release both dynamically allocated arrays.

With input count $==$ 3, the output should match Homework 2 and 3.

Compare the representations

Homework 2	Homework 3	Homework 4
pointer and length are separate	array stored inside structure	pointer stored inside structure
array allocated dynamically	fixed-size array	array allocated dynamically
use `free`	no `free`	use `free`

Why does Homework 4 require `free` while Homework 3 does not?

Submission

Submit your completed `vector_ops2.c` from Homework 4.

Before submitting, make sure your code compiles and runs.